PRACTICAL NO. 02 — CLASS-OBJECT, METHODS, CONSTRUCTORS

Problem Statement No. 1 Bank Account Management System	
Problem Statement	A private bank wants to model its savings account system in Java. Design a class 'BankAccount' with: • Member variables: accountNumber (int), holderName (String), balance (double) • Parameterized constructor to initialise all fields • Methods: deposit(double amt), withdraw(double amt), displayBalance() • Method checkMinBalance() — returns a message if balance falls below ₹1000 Tasks: (a) Create at least 3 BankAccount objects with different initial balances (b) Perform deposit and withdrawal operations on each (c) Call checkMinBalance() and display appropriate message (d) Show overloaded method: deposit(double, String) where String is a remark
Sample Input	Acc1: 1001, 'Amit', 5000 | Acc2: 1002, 'Priya', 800 | deposit 2000 to Acc1 | withdraw 700 from Acc2
Expected Output	Updated balances, min-balance warning for Acc2 after withdrawal

Problem Statement No. 2 Employee Payroll Calculator	
Problem Statement	An IT company needs a payroll system. Design a class 'Employee' with: • Member variables: empId (int), name (String), basicSalary (double), department (String) • Default and parameterized constructors • Methods: calculateHRA() — 20% of basic, calculateDA() — 11% of basic, calculateNetSalary() — basic + HRA + DA, displayPayslip() • Overloaded method displayPayslip(boolean showDeductions) — if true, show tax (10% of net) Tasks: (a) Create 3 Employee objects using parameterised constructor (b) Display payslip for each employee (c) Display payslip with deductions for employees earning > ₹50,000 net (d) Demonstrate use of 'this' keyword inside constructor
Sample Input	E1: 201, 'Rohit', 40000, 'IT' | E2: 202, 'Sneha', 60000, 'HR' | E3: 203, 'Karan', 30000, 'Finance'
Expected Output	Payslips with HRA, DA, Net Salary; deductions shown for Sneha only

PRACTICAL NO. 03 — JAVA INHERITANCE

Problem Statement No. 3 University Staff Hierarchy	
Problem Statement	A university maintains different categories of staff. Model the hierarchy using Java Inheritance: • Base class 'Person': name, age, contactNo; method displayInfo() • Class 'Employee' extends Person: empId, department; method displayEmployeeDetails() • Class 'Faculty' extends Employee: designation, subjectsTaught[]; method displayFacultyDetails() • Class 'HodFaculty' extends Faculty: additionalAllowance; method displayHodDetails() Tasks: (a) Demonstrate Single (Employee → Person), Multilevel (HodFaculty → Faculty → Employee → Person), and Hierarchical inheritance (Faculty & AdminStaff both extending Employee) (b) Create objects for each type and call respective display methods (c) Use 'super' to call parent constructor from child class

Sample Input	Faculty: 'Dr. Mehta', 45, F001, 'IT', 'Professor', ['Java','OS'] | HoD allowance: 5000
Expected Output	Complete hierarchy details printed for each object using appropriate display methods

Problem Statement No. 4	**E-Commerce Product Catalogue**
Problem Statement	An e-commerce platform sells different product categories. Model this with Java Inheritance: • Base class 'Product': productId, name, price; method displayProduct() • Class 'Electronics' extends Product: brand, warrantyYears; method displayElectronics() • Class 'Clothing' extends Product: size, material; method displayClothing() • Class 'Grocery' extends Product: expiryDate, weightKg; method displayGrocery() Tasks: (a) Demonstrate Hierarchical inheritance (Electronics, Clothing, Grocery all extend Product) (b) Create 2 objects of each subclass with different details (c) Override the displayProduct() method in each subclass to show category-specific info (d) Call the parent version using 'super.displayProduct()' within each override
Sample Input	Phone: P001, 'Samsung', 25000, 'Samsung', 2yrs | Shirt: C001, 'Blue Shirt', 799, 'M', 'Cotton'
Expected Output	Product details for all 6 objects with category-specific fields

PRACTICAL NO. 04 — STATIC, FINAL, SUPER AND THIS KEYWORD

Problem Statement No. 5	**College Admission Counter System**
Problem Statement	A college admission office tracks the number of students admitted. Design a Java program: • Class 'Student': name, rollNo, branch; static variable totalAdmitted (auto-incremented in constructor) • final variable MAX_CAPACITY = 60 (cannot change once set) • Constructor uses 'this' keyword to assign values; calls super() if Student extends Person • Static method getTotalAdmitted() returns current count • Method displayStudent() shows all details Tasks: (a) Create 5 Student objects and verify totalAdmitted increments correctly (b) Try reassigning MAX_CAPACITY — show and explain the compile error (comment it) (c) Show use of 'this()' constructor chaining for a default constructor (d) Show 'super' calling parent class (Person) constructor
Sample Input	5 students: Asha/CS, Raj/IT, Priya/CS, Dev/ENTC, Neha/IT
Expected Output	Each student's details + Total Admitted = 5; MAX_CAPACITY unchanged at 60

Problem Statement No. 6	**Vehicle Registration System**
Problem Statement	A Regional Transport Office (RTO) manages vehicle registrations. Design a Java program: • Base class 'Vehicle': vehicleType (final), engineCC (int); constructor uses 'super' from subclass • Class 'RegisteredVehicle' extends Vehicle: regNo, ownerName; static int totalRegistered • Static method getTotalRegistered(); instance method displayRegistration() • final method getVehicleType() in Vehicle — cannot be overridden Tasks: (a) Create 4 RegisteredVehicle objects; verify static count increases (b) Attempt to override getVehicleType() in

	subclass — explain final method restriction (comment) (c) Use 'this' to differentiate instance variable from parameter in constructor (d) Use 'super(vehicleType, engineCC)' in RegisteredVehicle constructor
Sample Input	Car/1500cc/MH12AB1234/Suresh | Bike/150cc/MH12CD5678/Pooja | Truck/3000cc | Van/2000cc
Expected Output	Registration details for all 4 vehicles; Total Registered = 4

PRACTICAL NO. 05 — POLYMORPHISM

Problem Statement No. 7	**Banking Transaction Processor**
Problem Statement	A banking application needs to process multiple types of transactions. Implement Polymorphism: Compile-time (Method Overloading): • Class 'Transaction': overloaded method processPayment(int), processPayment(double), processPayment(double, String) where String is remarks, processPayment(String upiId, double amt) Runtime (Method Overriding): • Base class 'Account': method calculateInterest() • Class 'SavingsAccount' extends Account: override to compute 4% p.a. interest • Class 'FixedDeposit' extends Account: override to compute 7% p.a. interest Tasks: (a) Demonstrate all 4 overloaded processPayment() calls with different arguments (b) Create base class reference pointing to SavingsAccount and FixedDeposit objects (c) Call calculateInterest() via base class reference — show dynamic dispatch (d) Print which account type is being used at runtime
Sample Input	Payment: 5000 | 2500.50 | 1000.0/'EMI' | 'upi@bank'/3000 | SA: 50000 | FD: 1,00,000
Expected Output	Overloaded method outputs; Interest: SA=2000, FD=7000; Runtime type shown

Problem Statement No. 8	**Smart Home Device Control**
Problem Statement	A smart home app controls different devices. Implement Polymorphism in Java: Compile-time (Overloading): • Class 'DeviceController': overloaded method setTimer(int minutes), setTimer(int hours, int minutes), setTimer(String schedule) Runtime (Overriding): • Base class 'SmartDevice': method activate() • Class 'SmartLight' extends SmartDevice: override activate() — "Turning lights ON at X% brightness" • Class 'SmartAC' extends SmartDevice: override activate() — "Setting AC to Y°C" • Class 'SmartFan' extends SmartDevice: override activate() — "Fan speed set to Z" Tasks: (a) Demonstrate all 3 overloaded setTimer() calls (b) Create SmartDevice array, store SmartLight, SmartAC, SmartFan objects (c) Loop through array, call activate() — show polymorphic behaviour (d) Use instanceof to identify device type at runtime
Sample Input	Timer: 30 | 1hr 30min | 'Morning 6AM' | Light: 75% | AC: 22°C | Fan: Speed 3
Expected Output	Timer outputs; loop prints correct activate() message for each device

PRACTICAL NO. 06 — ABSTRACT CLASS, INTERFACES, LAMBDA EXPRESSION

Problem Statement No. 9	Insurance Premium Calculator
Problem Statement	An insurance company offers different policies. Model using Abstract Class, Interface and Lambda: • Abstract class 'InsurancePolicy': policyNo, holderName, sumInsured; abstract method calculatePremium(); concrete method displayPolicy() • Class 'HealthInsurance' extends InsurancePolicy: implements calculatePremium() as 2% of sumInsured • Class 'VehicleInsurance' extends InsurancePolicy: implements calculatePremium() as 3% of sumInsured • Interface 'Taxable': method applyGST(double premium) — returns premium + 18% GST • Both classes implement Taxable Lambda: • Use lambda to sort a list of InsurancePolicy objects by sumInsured (ascending) • Use lambda with Predicate<InsurancePolicy> to filter policies where premium > ₹5000 Tasks: (a)–(d) Create 3 health and 2 vehicle policies, display all, sort by sumInsured, filter high-premium
Sample Input	Health: H001/Asha/2,50,000 | H002/Raj/5,00,000 | Vehicle: V001/Priya/1,50,000
Expected Output	Premiums with GST, sorted list, filtered list (premium > 5000)

Problem Statement No. 10	Food Delivery Order System
Problem Statement	A food delivery app (like Swiggy) needs to model its order processing. Implement in Java: • Abstract class 'FoodItem': itemName, price; abstract method prepareItem(); concrete displayItem() • Class 'VegItem' extends FoodItem: implements prepareItem() — "Preparing Veg: <name>" • Class 'NonVegItem' extends FoodItem: implements prepareItem() — "Preparing Non-Veg: <name>" • Interface 'Discountable': method applyDiscount(double percent) — returns discounted price • Interface 'Trackable': method trackOrder() — returns estimated delivery time string • Both VegItem and NonVegItem implement both interfaces Lambda: • Use lambda with Comparator to sort food items by price • Use lambda with forEach to print item name and final price after discount Tasks: (a) Create 4 food items (mix veg/non-veg), call prepareItem(), applyDiscount(10%), trackOrder() (b) Sort by price using lambda; print final menu using lambda forEach
Sample Input	Paneer Butter Masala/320/Veg | Chicken Biryani/280/NonVeg | Dal Tadka/180/Veg | Mutton Curry/380/NonVeg
Expected Output	Preparation messages, discounted prices, delivery ETA, sorted menu

PRACTICAL NO. 07 — EXCEPTION HANDLING

Problem Statement No. 11	ATM Withdrawal System
Problem Statement	An ATM system must handle errors gracefully. Implement Exception Handling in Java: Custom Exceptions: • InsufficientFundsException (checked) — thrown when withdrawal > balance • InvalidAmountException (unchecked) — thrown when withdrawal amount ≤ 0 or not multiple of 100 • DailyLimitExceededException (checked) — thrown when total daily withdrawal > ₹20,000 Class 'ATM': • Method withdraw(double amount) throws

	InsufficientFundsException, DailyLimitExceededException • Validates amount using InvalidAmountException (unchecked — no throws clause needed) • finally block: always prints "Transaction log saved." Tasks: (a) Perform 4 withdrawals: valid, insufficient funds, invalid amount (–500), daily limit breach (b) Catch each exception separately and print a user-friendly message (c) Show use of try-with-multiple-catch and finally (d) Re-throw InsufficientFundsException after logging
Sample Input	Balance: 15000, Daily limit: 20000 | Withdrawals: 5000, 12000 (limit breach), –500 (invalid), 8000 (insufficient)
Expected Output	Success for ₹5000; appropriate exception messages for others; 'Transaction log saved.' always printed

Problem Statement No. 12	**Student Online Exam Portal**
Problem Statement	An online exam portal must validate student submissions. Implement Exception Handling: Custom Exceptions: • ExamTimeExpiredException (checked) — thrown if submission time > allowed duration • InvalidAnswerFormatException (unchecked) — thrown if answer is null or empty • AlreadySubmittedException (checked) — thrown if student tries to submit twice Class 'ExamPortal': • Method submitExam(Student s, long timeTaken, String[] answers) — validates all conditions • Uses try-catch-finally: finally prints "Session closed for <studentName>" Tasks: (a) Simulate 4 submission attempts: valid, time expired (95 min for 90 min exam), invalid answer (null), already submitted (b) Catch exceptions separately with user-friendly error messages (c) Demonstrate creating and throwing custom checked exception with a message (d) Use multi-catch (|) syntax for AlreadySubmittedException | ExamTimeExpiredException
Sample Input	Student: Raj | Duration: 90 min | Attempts: valid(85min), timeOut(95min), null answer, double-submit
Expected Output	Exam submitted for valid; exception messages for others; 'Session closed for Raj' always printed

PRACTICAL NO. 08 — JAVA THREAD

Problem Statement No. 13	**Railway Ticket Booking — Seat Reservation**
Problem Statement	A railway booking system must handle multiple passengers booking simultaneously. Implement Multithreading: • Class 'SeatReservation': shared variable availableSeats = 10 (shared resource) • Class 'BookingAgent' extends Thread: each agent tries to book a specified number of seats • Use 'synchronized' method bookSeats(int count, String agentName) to prevent race condition • Show what happens WITHOUT synchronization (race condition) — comment that version Tasks: (a) Create 4 BookingAgent threads: Agent1 books 3, Agent2 books 4, Agent3 books 2, Agent4 books 3 (b) Run all threads; verify total booked never exceeds 10 (c) Print booking confirmation or "Seats not available" for each agent (d) Use Thread.sleep() to simulate network delay between bookings (e) Display thread name and seats booked using Thread.currentThread().getName()

Sample Input	AvailableSeats: 10 | Agent1:3, Agent2:4, Agent3:2, Agent4:3 (total requested: 12)
Expected Output	First 3 agents fully/partially served (total ≤ 10); Agent4 gets 'Seats not available'

Problem Statement No. 14	**Stock Market Price Feed Simulator**
Problem Statement	A stock market application updates prices of multiple stocks simultaneously. Implement using Threads: • Class 'StockUpdater' implements Runnable: stockName, price (volatile); run() simulates price update in a loop (5 iterations) with random fluctuation ±5%; sleeps 500ms between updates • Class 'StockMonitor' extends Thread: monitors a specific stock, prints alert if price crosses threshold • Main thread creates 3 StockUpdater threads (RELIANCE, INFY, TCS) and 1 StockMonitor Tasks: (a) Start all threads simultaneously; each updater prints updated price every 500ms (b) StockMonitor prints "ALERT: <stock> crossed ₹<threshold>!" when condition met (c) Use join() to wait for all updater threads before printing final prices (d) Demonstrate Runnable interface vs Thread class — use both in this program (e) Print thread states using Thread.getState()
Sample Input	RELIANCE start: 2500, threshold 2600 | INFY: 1800 | TCS: 4000 | 5 update iterations
Expected Output	Live price feed for all stocks; alert if RELIANCE > 2600; final prices after all threads finish

PRACTICAL NO. 09 — PACKAGES AND COLLECTIONS

Problem Statement No. 15	**Hospital Patient Record System**
Problem Statement	A hospital manages patient records using Java packages and collections. Design as follows: Package Structure: • Package 'hospital.model': class Patient (patientId, name, age, disease) • Package 'hospital.service': class PatientService (business logic methods) • Package 'hospital.main': Main class importing and using above packages Collections (in PatientService): • ArrayList<Patient>: store all patients; add, remove, display • HashMap<String, Patient>: map patientId → Patient for O(1) lookup • LinkedList<Patient>: maintain queue for doctor consultation (FIFO) • TreeSet<String>: store unique disease names in sorted order Tasks: (a) Add 5 patients across packages; display all using ArrayList (b) Search patient by ID using HashMap (c) Simulate next-in-queue using LinkedList poll() (d) Display all unique diseases alphabetically using TreeSet
Sample Input	P001/Asha/32/Fever | P002/Raj/45/Diabetes | P003/Priya/28/Fever | P004/Dev/60/BP | P005/Neha/35/Asthma
Expected Output	All patient list, ID lookup result, next patient from queue, sorted disease list

Problem Statement No. 16	**Online Shopping Cart System**
Problem Statement	An online store needs a shopping cart and product inventory system using Java packages and collections: Package Structure: • Package 'store.model': class

	Product (productId, name, price, category) • Package 'store.cart': class ShoppingCart (manages cart operations) • Package 'store.main': Main class Collections (in ShoppingCart): • HashMap<String, Integer>: productId → quantity in cart • ArrayList<Product>: full product catalogue • Stack<String>: undo history (last added item can be removed) • TreeMap<Double, String>: sorted price → productName for price-range browsing Tasks: (a) Add 6 products to catalogue; add 4 to cart with quantities (b) Remove last added item using Stack (undo feature) (c) Display cart total (d) Display products sorted by price using TreeMap (e) Search product by ID using HashMap
Sample Input	Products: Phone/25000, Laptop/65000, Earphones/2000, Mouse/800, Keyboard/1500, Monitor/12000
Expected Output	Cart details, undo action, total bill, price-sorted product list

PRACTICAL NO. 10 — FILE HANDLING

Problem Statement No. 17	**Student Result File Management**
Problem Statement	A university's exam department manages results using file handling. Implement in Java: Tasks: (a) List all .txt files present in a given directory path (e.g., results/) using File class (b) Write student results to 'results.txt': Format per line — RollNo, Name, Marks, Grade (compute grade: ≥60=B, ≥75=A, ≥90=O, else F) (c) Read 'results.txt' line by line using BufferedReader; display formatted result table (d) Append a new result record to existing file without overwriting (e) Count total number of students who passed (marks ≥ 40) by reading the file Use: File, FileWriter, BufferedWriter, FileReader, BufferedReader, FileNotFoundException handling
Sample Input	Directory: results/ (contains 3 files) | Students: 101/Asha/85, 102/Raj/72, 103/Priya/91, 104/Dev/38 New append: 105/Neha/65
Expected Output	File list, results.txt content as table, append confirmation, pass count = 4

Problem Statement No. 18	**Daily Expense Tracker Log**
Problem Statement	A personal finance app logs daily expenses into a file. Implement using Java File Handling: Tasks: (a) List all files in a given 'expenses/' directory and display names with sizes (b) Write 5 expense records to 'expenses_log.txt': Format — Date, Category, Amount, Description (one record per line) (c) Read 'expenses_log.txt' line by line; display in formatted table (d) Append today's new expense without overwriting existing records (e) Read file and calculate: • Total expenditure • Maximum single expense • Count of expenses in category "Food" Use: File, FileWriter (append=true), BufferedReader, PrintWriter, proper exception handling
Sample Input	Expenses: 01-May/Food/250/Lunch | 01-May/Travel/80/Bus | 02-May/Food/310/Dinner | 02-May/Shopping/1200/Shirt | 03-May/Food/190/Breakfast | Append: 03-May/Travel/50/Auto
Expected Output	File listing, formatted expense table, total=2080, max=1200, Food count=3

PRACTICAL NO. 11 — JAVA SWING

Problem Statement No. 19	Student Registration Form (GUI)
Problem Statement	Design a Student Registration GUI application using Java Swing: UI Components Required: • JTextField: Student Name, Email, Contact No., City • JRadioButton: Gender (Male / Female / Other) — in ButtonGroup • JCheckBox: Courses of Interest (Java, Python, Web Dev, Data Science) • JComboBox: Branch (CS, IT, ENTC, Mechanical, Civil) • JList: Hobbies (multi-select: Reading, Sports, Music, Coding, Travel) • JTextArea: Address (with JScrollPane) • JButton: Submit, Clear, Exit • Layout: GridBagLayout or BorderLayout + inner GridLayout On Submit: • Validate: Name and Email cannot be empty (show JOptionPane warning) • Display all entered details in a JTextArea below or in a new dialog • On Clear: reset all fields to default • On Exit: JOptionPane confirm before closing
Sample Input	GUI interaction: fill form with Name=Asha, IT branch, Java+Python checked, Female, submit
Expected Output	Validation message if empty; summary of registration details in dialog

Problem Statement No. 20	Library Book Search and Issue GUI
Problem Statement	Design a Library Book Search and Issue System using Java Swing: UI Components Required: • JTextField: Search by Book Title or Author • JButton: Search, Issue Book, Return Book, Clear • JTable (with DefaultTableModel): display book list — BookID, Title, Author, Status • JLabel: display search result count • JComboBox: Filter by Category (Fiction, Science, History, Technology, All) • JMenuBar with JMenu: File (Exit), Help (About) • JScrollPane wrapping JTable • JOptionPane: confirmations for Issue/Return actions On Search: filter JTable rows matching title/author (case-insensitive) On Issue: change selected book Status to "Issued" On Return: change status back to "Available" Pre-load: at least 6 books in the table
Sample Input	Search 'Java' → filter results; select 'Core Java' book; click Issue
Expected Output	Table filtered; status changes to 'Issued'; confirmation dialog shown

PRACTICAL NO. 12 — JDBC

Problem Statement No. 21	Employee Records Management using JDBC
Problem Statement	An HR system stores employee records in a MySQL database. Implement a JDBC application: Database: 'company_db' Table: Employee (EmpId INT PK, Name VARCHAR, Department VARCHAR, Salary DOUBLE, JoiningDate DATE) Tasks: (a) CREATE the Employee table (drop if exists first) (b) INSERT 5 employee records using PreparedStatement (c) SELECT and DISPLAY all records using ResultSet (formatted output) (d) INSERT 2 more employees using batch execution (addBatch / executeBatch) (e) UPDATE salary of employee with EmpId = 2 by increasing 15% (f) DELETE the employee with EmpId = 5 (g) SELECT employees where Department = 'IT' and display Use: DriverManager, Connection, PreparedStatement, ResultSet, SQLException handling

Sample Input	E1:101/Asha/IT/45000 | E2:102/Raj/HR/38000 | E3:103/Priya/IT/52000 | E4:104/Dev/Finance/41000 | E5:105/Neha/IT/47000 | Batch: E6,E7
Expected Output	All records displayed, batch insert confirmed, Raj salary updated, Neha deleted, IT dept list

Problem Statement No. 22	**Library Book Management using JDBC**
Problem Statement	A library system manages books and issue records using JDBC and MySQL. Implement: Database: 'library_db' Table 1: Books (BookId INT PK, Title VARCHAR, Author VARCHAR, Category VARCHAR, Available INT) Table 2: IssuedBooks (IssueId INT PK, BookId INT FK, StudentName VARCHAR, IssueDate DATE, ReturnDate DATE) Tasks: (a) CREATE both tables (drop if exists) (b) INSERT 5 books using PreparedStatement (c) DISPLAY all books using ResultSet (d) Issue a book: INSERT into IssuedBooks AND UPDATE Books.Available = 0 (use transaction — commit/rollback) (e) Return a book: UPDATE IssuedBooks.ReturnDate AND UPDATE Books.Available = 1 (transaction) (f) DELETE a book by BookId (g) DISPLAY books by Category using parameterised query Use: Connection, PreparedStatement, transaction management (setAutoCommit, commit, rollback)
Sample Input	Books: B001/Core Java/Schildt/Tech | B002/DBMS/Korth/Tech | B003/Fiction/Chetan/Fiction | Issue: B001 to 'Asha' on today's date | Return: IssueId=1 | Delete: B003
Expected Output	Book list, issue transaction committed, return updated, B003 deleted, Tech category list

PRACTICAL NO. 13 — JAVA GENERICS

Problem Statement No. 23	**Generic Product Inventory Comparator**
Problem Statement	A product inventory system uses Java Generics for type-safe operations. Implement: (a) Generic method checkIdentical(T[] arr1, T[] arr2): compares two arrays of same type element-by-element and returns true if identical in order — use this with Integer[], String[], and Double[] arrays (b) Generic method sumByParity(List<? extends Number> numbers): returns a String "Even Sum: X, Odd Sum: Y" (treat even/odd by int value of each number) Real-world scenario: (c) Generic class 'Inventory<T>': holds a list of items; methods add(T item), remove(T item), display(), contains(T item) (d) Instantiate Inventory<String> for product names, Inventory<Integer> for product codes, Inventory<Double> for prices — demonstrate type safety (e) Show that Inventory<String> cannot hold an Integer (compile-time error — comment it)
Sample Input	arr1=[10,20,30] arr2=[10,20,30] → identical | arr1=[10,20,30] arr2=[10,25,30] → not identical numbers=[1,2,3,4,5,6] → Even Sum:12, Odd Sum:9
Expected Output	Comparison results, parity sums, inventory operations for all 3 types

Problem Statement No. 24	**Generic Student Grade Analyser**

Problem Statement	An academic analytics system uses Java Generics for type-safe grade processing. Implement: (a) Generic method compareRecords(T[] batch1, T[] batch2): checks if two batches have identical records in same order — test with String[] (names), Integer[] (roll numbers), Double[] (marks) (b) Generic method analyseNumbers(List<? extends Number> marks): compute and return "Even-valued marks sum: X, Odd-valued marks sum: Y" Real-world extension: (c) Generic class 'GradeBook<T extends Comparable<T>>': stores grades; methods: add(T), getMax(), getMin(), displayAll() (d) Demonstrate GradeBook<Integer> for marks and GradeBook<Double> for GPA values (e) Generic method swap(T[] arr, int i, int j): swaps two elements; test on Integer[] and String[]
Sample Input	batch1=["Asha","Raj","Priya"] batch2=["Asha","Raj","Dev"] → not identical marks=[72,85,90,63,48,76] → Even:72+90+48=210, Odd:85+63+75=223 GradeBook: 85,72,91,63 → Max=91, Min=63
Expected Output	Comparison result, parity analysis, GradeBook max/min, swapped arrays

---

Instructions for Students

1. The examiner will assign one problem statement per student by draw of lots.
2. Students must write and execute the complete Java program within 2 hours.
3. Code must compile and run without errors; partial working code attracts partial marks.
4. Students must demonstrate program execution and explain the logic to the examiner.
5. Use of pre-written code, internet browsing, or mobile phones is strictly prohibited.
6. For JDBC problems (21–22), MySQL server must be running; credentials will be provided.
7. For Swing problems (19–20), GUI must be functional with all components responding to events.
8. Generics programs must demonstrate type safety with at least 2 different type arguments.